Workshop 8: Disks and Files

Remember: Discuss each question with your parnter and write down a short answer. Writing is important to force you to really think about whether your analysis was right when you do the experiment later. *Downloads:* See the Workshop info on the class website. *WSL and compiling:* See Workshop 1 for how to copy files, use WSL, and compile. (Or ask the TA, and if they are very nice, they may help you remember.)

For the workshop you should work with your partner at your own pace. Try to get as much of the workshop done as you can, but there is no requirement to finish it all.

If you need help or have questions, feel free to ask other groups or the TA. Don't forget to sign in for attendance before you leave.

Part 1: Devices, interrupts, and DMA

Question 0: Looking for DMA's effect on your own laptop

Even without special tools, we can see indirect evidence of DMA: if the CPU had to copy every byte of a large file itself, copying gigabytes of data would keep it almost 100% busy for the whole operation. With DMA, the CPU mostly just waits (or does something else) while the disk controller moves the bytes. Let's test this.

Note that the system caches files, so the performance numbers you get here may be different on different machines.

- Create a large (~2GB) file: `dd if=/dev/zero of=bigfile bs=1m count=2048` (macOS/Linux/WSL).
- Discuss with your partner: if the CPU has to move ever byte itself, how busy will the CPU be for the time it takes to copy the file?
-
- Now copy it and time it: `time dd if=bigfile of=bigfile2 bs=1m cov=fsync`. Compare the "real/total" (actual time the whole thing took) time to the "user"+"sys" (CPU processing) time it reports.
-
- Does this match your prediction? What does the gap between "real" time and CPU time tell you about what the CPU was doing for the rest of that time?
-
- The CPU needs a fixed amount of time to setup each DMA command. Discuss with your partner what would happen to the CPU time if you split up the transfer into more smaller blocks. Would it go up or down?
-
- Now change the blocksize (`bs`) that is used for the copies. Try it with a block size of 1024 (1 KB), 10240 (10 KB), 102400 (100KB) and compare it to the previous 1m (1024 KB). The smaller block sizes mean the CPU needs to do more transfers.
- Do these results match with your assessment of the DMA overhead for more transfers?
- Clean up: `rm bigfile bigfile2`.

Part 1: File system on-disk state (`vsfs.py`)

Preparation

- Download the code for `file-implementation/vsfs.py` from the class website (already provided in this workshop's folder) or the book website.
- Optional: Read the "Overview" section of `file-implementation/README.md`, up through the description of the four data structures (inode bitmap, inodes, data bitmap, data).

Question 0: Reading the on-disk state

- Let's first look at how the simulator displays the file system state.

Initial state:

```
inode bitmap  10000000
inodes       [d a:0 r:2][][][][][][][]
data bitmap   10000000
data         [(.,0) (..,0)][][][][][][][]
```

- What do the two bitmaps represent?
-
- What do you think the contents of the first data block `(.,0) (..,0)` represents? (Hint: what part of the file system has . and .. as the first two entries?)
-
- What does the `d` in the first inode indicate about its type?
-
- How can you see that inode 0 points to data block 0?
-
- The standard unix convention is that empty directories have two links (references): the one in its parent directory and the `.` in its own directory. Where do you see this in the example?
-
- One thing to remember: when a new file is created in this simulator it is empty, so its data block address will be `a:-1`.

Question 1: What operations happened?

Below are the file system's states after each of six operations, generated with a fixed seed so everyone in your pair sees the same sequence. **Do not run the simulator yet** - work out what operation happened between each pair of states first, then check yourselves.

- The operations to consider are create(), link(), unlink(), mkdir(), and open(). Note that open() will allocate a data block for a file if it doesn't have one already.

Initial state:

```
inode bitmap  10000000
inodes       [d a:0 r:2][][][][][][][]
data bitmap   10000000
data         [(.,0) (..,0)][][][][][][][]
```

- Same as above.

After operation 1:

```
inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (n,1)][(.,1) (..,0)][][][][][][][]
```

- What operation took place?
-

After operation 2:

```
inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (n,1) (w,2)][(.,1) (..,0)][][][][][][][]
```

- What operation took place?
-

After operation 3:

```
inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (n,1)][(.,1) (..,0)][][][][][][][]
```

- What operation took place? (Hint: inode 2 is gone, and it was never linked from anywhere else.)
-

After operation 4:

```
inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (n,1)][(.,1) (..,0) (x,2)][][][][][][][]
```

- What operation took place? (Hint: which directory did the new entry appear in?)
-

After operation 5:

```
inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:2 r:1][][][][][][]
data bitmap   11100000
data          [(.,0) (..,0) (n,1)][(.,1) (..,0) (x,2)][o][][][][][][]
```

- What operation took place?
-

After operation 6:

```
inode bitmap  11110000
inodes       [d a:0 r:3][d a:1 r:2][f a:2 r:1][f a:-1 r:1][][][][][]
data bitmap   11100000
data         [(.,0) (..,0) (n,1) (m,3)][(.,1) (..,0) (x,2)][o][][][][][][]
```

- What operation took place?
-
- Now run it yourself to check every step: `python3 vsfs.py -n 6 -s 1 -c`.
- Did you get every one right? Which one was hardest to spot, and why?

Part 3: Crash consistency (`fsck.py`)

Preparation

- Download the code for `file-journaling/fsck.py` from the class website (already provided in this workshop's folder) or the book website.
- Optional: Read the "Overview" section of `file-journaling/README.md` through its worked example.

Question 0: How does this simulator work?

- What does `-c` do, and what would `-D` do instead?
-
- The tool always introduces exactly one inconsistency at a time. Why does that make sense for practicing what a real `fsck` (or a journal replay) has to reason about?
-
- The root directory is always inode 0. Why does starting from the root and walking outward help you find an inconsistency, rather than just staring at the raw bitmaps and inode table?
-

Question 1: Corruption A

Run `./fsck.py -S 1` (do **not** add `-c` yet) and you should see this final state: (I've added spaces in the bitmaps to make them easier to count.)

```
inode bitmap 1000 1000 1000 0101
inodes       [d a:0 r:4] [] [] [] [d a:12 r:2] [] [] [] [d a:6 r:2] [] [] [] []
[f a:-1 r:2] [] [f a:-1 r:1]
data bitmap  1000 0010 0000 1000
data         [(.,0) (..,0) (g,8) (w,4) (m,13) (z,13)] [] [] [] [] [] [(.,8)
(..,0) (s,15)] [] [] [] [] [] [(.,4) (..,0)] [] [] []
```

- Starting from the root directory (inode 0, in data block 0), which names point at inode 13, and what does inode 13's entry tell you it should be? (In particular, what is the link/reference count `r` for the inode and what does inode 0 say it should be?)
-
- Now look at the inode bitmap. Is bit 13 (the 14th character, counting from 0) marked allocated or free? Does that match what you just found?
-
- Run it again with `-c` to confirm.

-
- If this went undetected and the OS later needed a free inode, what could go wrong?
-

Question 2: Corruption B

Run `./fsck.py -S 4` (still no `-c`) and you should see this final state: (I've added spaces in the bitmaps to make them easier to count.)

```
inode bitmap 1000 1000 1000 0101
inodes       [d a:0 r:4] [] [] [] [d a:12 r:2] [] [] [] [d a:6 r:2] [] [] [] []
[f a:-1 r:3] [] [f a:-1 r:1]
data bitmap  1000 0010 0000 1000
data         [(.,0) (..,0) (g,8) (w,4) (m,13) (z,13)] [] [] [] [] [] [(.,8)
(..,0) (s,15)] [] [] [] [] [] [(.,4) (..,0)] [] [] []
```

- This is the *same* file system as Question 1 with a different corruption. Is the inode bitmap consistent this time?
-
- Count, by hand, how many directory entries anywhere actually point at inode 13. Compare that to inode 13's own reference count field.
-
- Run it with `-c` to confirm.
-
- Suppose this goes undetected and someone later deletes both `/m` and `/z`. What would go wrong, and how is that different from the danger in Question 1?
-

Question 3: Corruption C

- Run `./fsck.py -S ?` (still no `-c`) with another seed instead of ?.
- Can you determine the corruption?

Synthesis: comparing the two corruptions

- Which of the first two corruptions could you have caught with a much simpler check - just scanning the two bitmaps and inode table directly, without walking any directories? Which one required you to actually count directory entries?
-

Part 4: Flash-based SSDs (`ssd.py`)

Preparation

- Download the code for `file-journaling/fsck.py` from the class website (already provided in this workshop's folder) or the book website.
- Optional: ead the "Overview" section of `file-ssd/README.md` up through the first `-C`/`-F` example (you do not need to read the garbage-collection section yet - we'll get to that).

Question 0: What do the flags do?

- What do the three operations `w`, `r`, and `t` in a workload string (e.g., `w10:a,r10,t10`) do? (hint: write, read, trim… what are `a` and 10?)
-
- What do `-l`, `-B`, and `-p` control? What would you set them to to get 30 logical pages with 3 physical blocks of 10?
-
- What do `-C` and `-F` each show you that plain output does not?
-
- What does the logical-to-physical mapping in the Flash Translation Layer (FTL) of 10:2, 11:3 mean?
-
- `-T ideal` runs the simulator in an ideal mode where every page can be erased instantly and independently. This means that the FTL always maps logical page X directly to physical page X. Why is this unrealistic?
-
- `-T direct` maps each logical page to the same physical page, but simulates erasing and copying as needed. Why would this be bad for performance?
-
- What do the three page states `i`, `E`, and `v` mean?
-

Question 1: Predict, then check, the simplest possible write

- Before running anything, based on the flag meanings above: if you run `./ssd.py -T ideal -L w10:a -l 30 -B 3 -p 10`. This is doing one write to logical block 10 of the data `a` using the `ideal` simulation. What will the FTL contain at the end, and what will physical page 10′s state and data be?
-
- Run it and check.
- Now do the same prediction, but for `-T direct` instead of `-T ideal`, with the *same* command otherwise. Will the FTL entry be any different?
-

Question 2: How inefficient is "direct" mapping, really?

The book explains that a "direct" SSD must read the whole block, erase it, and reprogram it (old data + new data) for every single logical write that lands on an already-used block, whereas a log-structured SSD just appends. Let's measure the difference directly instead of just reading about it.

- Run all three commands below (same random workload, same random seed, so the only difference is the SSD type) and record the total time reported at the bottom (`-S`):

Command	Erases	Writes	Total time
`./ssd.py -T ideal -l 30 -B 3 -p 10 -s 10 -n 60 -S`			
`./ssd.py -T direct -l 30 -B 3 -p 10 -s 10 -n 60 -S`			
`./ssd.py -T log -l 30 -B 3 -p 10 -s 10 -n 60 -S`			

-
- Why does "ideal" need zero erases at all, while both "direct" and "log" need some? (Hint: is this realistic?)
-
- "Direct" does about 4x as many physical writes as logical writes were issued (171 vs. 37 write-count). Why? (Hint: what else does it have to rewrite besides the one page you asked it to write?)
-
- "Log" is much closer to "direct" in erase count than in total time. Where does most of "log"'s remaining cost come from, given it never has to move already-live data out of the way before writing?
-

Question 3: Where to write?

So far we've only looked at final states. Let's build one up ourselves and predict what the *next* physical operation has to do. - Run this, which writes to each of teh first 10 logical pages once, in-order: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w0:a,w1:b,w2:c,w3:d,w4:e,w5:f,w6:g,w7:h,w8:i,w9:j` - You should see:

```
FTL     0:  0   1:  1
        2:  2   3:  3
        4:  4   5:  5
        6:  6   7:  7
        8:  8   9:  9

Block 0          1
Page  0000000000 1111111111
      0123456789 0123456789
State vvvvvvvvvv iiiiiiiiii
Data  abcdefghij
Live  ++++++++++
```

- Why is the FTL perfectly ordered?
-
- What will the FTL look like if the order of write logical addresses are reversed? (E.g., w9:a, w8:b, etc.)
-
- What will the order of the data in the SSD look like in that case? (E.g., the `abcdefghij`)?
-

- Now try this and see what happens: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w9:a,w8:b,w7:c,w6:d,w5:e,w4:f,w3:g,w2:h,w1:i,w0:j`
- Why are the data values that way despite changing the order?
-
- What will happen when we add one more write: `w10:k`? Can this write be done?
-
- Try it and confirm: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w0:a,w1:b,w2:c,w3:d,w4:e,w5:f,w6:g,w7:h,w8:i,w9:j,w10:k`
- What would happen if instead we did one more write to the first logical block? (e.g., replacing the `a` from the start with `A`) What needs to be done to make that work?
-
- Try it and confirm: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w0:a,w1:b,w2:c,w3:d,w4:e,w5:f,w6:g,w7:h,w8:i,w9:j,w0:A`
- Now, what happens if we instead write-over the whole first 10 logical addresses with `ABCDEFGHIJ` and then do one more write `X` to location 10? What will the state of the SSD be after the first 10 are overwritten with `ABCDEFGHIJ`? What will the SSD have to do before it can write the `X`? Where will the `X` go?
-
- Try it out writing in `ABCDEFGHIJ`: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w0:a,w1:b,w2:c,w3:d,w4:e,w5:f,w6:g`
- Now what will happen if we write `X` to 0? Where can it go? What do you need to do before you put it in?
-
- Try it out: `./ssd.py -T log -l 20 -B 2 -p 10 -C -L w0:a,w1:b,w2:c,w3:d,w4:e,w5:f,w6:g,w7:h,w8:i,w9:j,w0:`
- (Optional) What would happen if you change the SSD to have 3 blocks? Can you configure the garbage collector (see below) to fix this?

Question 4: (Optional) Tuning the garbage collector

Log-structured SSDs need garbage collection (GC) to reclaim blocks whose data is no longer all live. The `-G` (high water mark) flag says "start GC once this many blocks are in use"; `-g` (low water mark) says "keep cleaning until only this many blocks are in use." A bigger gap between `-G` and `-g` means each GC pass reclaims more space at once.

- Before running anything: cleaning *more* (a lower `-g`) means the GC has to move more still-live data out of the blocks it reclaims each time it runs. Would you expect that to cost more or less physical work overall than cleaning just a little each time?
-
- Run both commands below and record the results:

Command	Erases	Writes (physical)	Total time
`./ssd.py -T log -l 40 -B 6 -p 10 -s 3 -n 200 -G 6 -g 5 -S` (`-g 5` = clean a little each time)			
`./ssd.py -T log -l 40 -B 6 -p 10 -s 3 -n 200 -G 6 -g 2 -S` (`-g 2` = clean a lot each time)			

-
- Was your prediction right? Cleaning *further* down every time it runs (`-g 2`) turns out to cost *more* overall, not less. Why? (Hint: what has to happen to every still-live page inside a block before that block can be erased and reused, no matter which watermark setting you use?)
-
- This is an example of the book's "spare space" trade-off. In your own words, why would giving the SSD *more* unused physical capacity (more spare blocks beyond what the logical size needs) tend to reduce this cost, at the expense of something else?
-

You're done for the day! *Don't forget to sign the attendance sheet.*